

1. User Secrets

What are User Secrets in ASP.NET Core?

A local configuration provider used to store sensitive key-value pairs outside the project root during local development.

What problem do User Secrets solve?

They prevent sensitive credentials (e.g., local database passwords, API keys) from being accidentally committed into source control systems like Git.

Where are User Secrets stored on Windows, Linux, and macOS?

- Windows: %APPDATA%\Microsoft\UserSecrets\<UserSecretsId>\secrets.json
- Linux / macOS: ~/.microsoft/usersecrets/<UserSecretsId>/secrets.json

Are User Secrets encrypted? Are they considered a trusted production secret store?

No, they are stored in plain JSON and are not encrypted. They are not a production secret store.

How is a project connected to its User Secrets store?

Linked via the <UserSecretsId> property inside the project's .csproj file.

CLI Commands for Managing Secrets

Action	Command
--------	---------

Initialize	dotnet user-secrets init
------------	--------------------------

Set	dotnet user-secrets set "Key" "Value"
-----	---------------------------------------

List	dotnet user-secrets list
------	--------------------------

Remove	dotnet user-secrets remove "Key"
--------	----------------------------------

Clear	dotnet user-secrets clear
-------	---------------------------

How does an ASP.NET Core application read a secret through IConfiguration?

Read normally via `builder.Configuration["Key"]` or `builder.Configuration.GetConnectionString("Key")`.

When should User Secrets be used?

During local development only, to isolate sensitive developer settings.

When should User Secrets not be used?

In non-development environments (Staging, Production) or for sharing secrets across team members.

Why must User Secrets not contain production credentials?

They reside unencrypted on disk, allowing anyone with host machine access to read them.

2. Environment Variables

What is an environment variable?

A global dynamic value maintained by the operating system or container environment that running processes can read.

Who can create environment variables?

All of them can create environment variables (Operating System, Terminal, IDE, Container, Cloud Host).

How does ASP.NET Core load environment variables into IConfiguration?

Automatically loaded at startup using the built-in configuration provider `AddEnvironmentVariables()`.

How are hierarchical configuration keys represented in environment variables?

Colons (:) in JSON keys are replaced with a double underscore (__) (e.g., `ConnectionStrings__DefaultConnection`).

Setting Environment Variables Across Platforms

Platform	Command / Location
PowerShell	<code>\$env:KEY="Value"</code>
CMD	<code>set KEY=Value</code>
Bash	<code>export KEY=Value</code>
Visual Studio	Configured inside <code>launchSettings.json</code>
Docker	Set via ENV in Dockerfile or environment in docker-compose

Platform	Command / Location
----------	--------------------

Azure App Service Configured under App Settings

What is ASPNETCORE_ENVIRONMENT, and what values are normally used?

An environment variable defining the execution context. Standard values: Development, Staging, and Production.

What is the difference between an environment name and an environment variable?

An environment variable is a key-value pair, while an environment name is the string value assigned to ASPNETCORE_ENVIRONMENT (accessed via `IHostEnvironment.EnvironmentName`).

Do environment variables automatically encrypt secrets?

No, they are stored as plain text.

When are environment variables more suitable than User Secrets?

In CI/CD pipelines, Docker containers, and cloud runtime hosts.

What risks exist when sensitive values are stored directly in environment variables?

Risks include memory dumps, process inspection tools, or child process inheritance leaks.

3. appsettings Files and Configuration Sources

What is appsettings.json, and when is it loaded?

The main non-sensitive configuration file loaded at application startup.

What is appsettings.{Environment}.json?

Environment-specific configuration files that override base settings (e.g., `appsettings.Development.json`, `appsettings.Staging.json`, `appsettings.Production.json`).

Default Configuration Precedence

When the same key exists across multiple providers, the winning priority (highest to lowest) is:

- 1. Command-line arguments (*Highest priority*)**
- 2. Environment variables**
- 3. User Secrets (*Development environment only*)**

4. appsettings.{Environment}.json

5. appsettings.json (*Lowest priority*)

What kinds of safe settings belong in appsettings.json?

Log levels, feature flags, UI constants, and default pagination limits.

What values must never be committed to appsettings.json?

Database passwords, private keys, API secrets, and production tokens.

Should a connection string always be considered a secret?

- **Credentialed:** Contains explicit user IDs and passwords → Secret.
- **Non-Credentialed:** Uses Windows Authentication (Integrated Security=true) or local development instances → Lower security risk, though internal network details should still be guarded.

When should a developer use User Secrets instead of environment variables?

During local machine development to keep local OS environment variables unpolluted and isolated per project.

When should a deployment use environment variables or a managed secret store instead of User Secrets?

In all non-local deployment environments (Staging, Production) because User Secrets are unencrypted and restricted to local machine scopes.

IConfiguration vs. Options Pattern

- **IConfiguration:** Best for quick, ad-hoc key retrieval.
- **Options Pattern (IOptions<T>):** Preferred for strongly-typed configuration binding, validation, and dependency injection.

4. Kestrel Configuration

What is Kestrel, and what role does it play in an ASP.NET Core application?

A cross-platform, high-performance web server that receives raw HTTP requests and forwards them through the middleware pipeline.

What is a Kestrel endpoint?

A network address binding defined by an IP address and port combination.

Which properties can be configured for an endpoint?

Address, port, protocol, and certificate can all be configured.

Supported ways to configure Kestrel endpoints:

Via appsettings.json, the ASPNETCORE_URLS environment variable, CLI flag --urls, or C# code (builder.WebHost.ConfigureKestrel(...)).

Why might a team configure Kestrel in appsettings.json?

To allow port and protocol modifications per environment without recompiling application code.

When is appsettings.json not the best place for final production endpoint configuration?

When production endpoint structures and certificates are dynamically handled by cloud infrastructure, reverse proxies, or secret managers.

IP Address Binding Comparison

- **localhost:** Accepts connections only from the local loopback interface.
- **0.0.0.0:** Accepts connections on all available IPv4 interfaces (essential for Docker containers).
- **Specific IP:** Restricts incoming traffic to a specific network interface card (NIC).

How can separate HTTP and HTTPS endpoints be configured?

By defining distinct binding objects with individual ports and protocol settings in setup code or JSON configuration.

What happens when an HTTPS endpoint has no explicit certificate in Development?

Kestrel falls back to the default .NET Development Certificate.

Certificate Storage Locations

- **Path:** Inside appsettings.json.
- **Password:** Inside User Secrets or Environment Variables.

How does Kestrel hosting differ when IIS, Nginx, a load balancer, or an ingress terminates TLS?

The external proxy handles TLS decryption (TLS Termination) and passes plain unencrypted HTTP traffic to Kestrel over a secure internal network.

5. Logging, ILogger, and Serilog

What is application logging, and why is it required in production systems?

The process of recording runtime events to monitor application health, troubleshoot errors, and audit system behavior in production.

Research every .NET log level: Trace, Debug, Information, Warning, Error, Critical, and None. Give two realistic situations for each log level.

- Situations: Inspecting raw network socket byte arrays; tracking step-by-step request payload deserialization.
- **Debug:** Short-term internal state information for development.
 - Situations: Measuring local database query execution time; logging internal loop counter variables during testing.
- **Information:** Normal application flow and business events.
 - Situations: User successfully logged in; payment transaction completed successfully.
- **Warning:** Unexpected non-fatal events where execution continues.

- Situations: Failed login attempt; query response time taking longer than expected threshold.
- **Error:** Failure of the current request or action.
 - Situations: Database query timeout exception for a specific request; SMTP failure when attempting to send an email.
- **Critical:** System-wide catastrophic failure requiring immediate intervention.
 - Situations: Unhandled database cluster outage blocking all traffic; server running out of disk space or memory.
- **None:** Disables all log output completely.

What does a configured minimum log level mean?

A threshold setting that instructs the logging framework to discard any log entries below the specified severity level.

What is ILogger? What is ILogger?

ILogger is the core logging abstraction interface. ILogger<T> uses generic category names automatically derived from the class type T.

How is ILogger obtained through dependency injection?

By requesting ILogger<T> as a constructor parameter in the target class.

What is a logging provider? Research the built-in Console and Debug providers.

A component that writes log events to a specific destination.

- **Console:** Writes logs to the standard console output stream.
- **Debug:** Writes logs to the active debugger window using `System.Diagnostics.Debug.WriteLine`.

What is structured logging? Compare message templates with string interpolation.

- **Structured Logging:** Formatting log data as named key-value parameters rather than plain text strings.
- **Message Template (Recommended):** `logger.LogInformation("User {UserId} logged in", userId);` (Preserves structured key-value data).
- **String Interpolation (Avoid):** `logger.LogInformation($"User {userId} logged in");` (Converts data into a plain text string, losing query capabilities).

What information must never be written to logs?

Passwords, payment card numbers, private encryption keys, JWT tokens, and personally identifiable information (PII).

What is Serilog, and how does it integrate with Microsoft.Extensions.Logging?

A structured logging library that integrates seamlessly by replacing the default `ILoggerFactory` provider.

What is a Serilog sink?

An output destination for routing Serilog log events (e.g., Console, File, Seq, Elasticsearch).

Research how to write Serilog events to the console and a file.

Configured via sinks: `.WriteTo.Console()` and `.WriteTo.File("logs/log.txt")`.

Research daily rolling files, file-size limits, and retained-file limits.

- **Daily Rolling:** Automatically creates a new log file every 24 hours (`rollingInterval: RollingInterval.Day`).
- **File-Size Limit:** Restricts max byte size for individual log files (`fileSizeLimitBytes`).
- **Retained-File Limit:** Automatically deletes older log files once the maximum file count is reached (`retainedFileCountLimit`).

Research request logging in Serilog.AspNetCore.

Enabled via `app.UseSerilogRequestLogging()`, which collapses multiple verbose framework HTTP events into a single structured summary log per request.

When are local log files unsuitable, especially in containers or multiple application instances?

In containerized (Docker) or multi-instance environments where container storage is ephemeral and local log files cannot be aggregated across scaled instances.

1. Repository & Unit of Work Patterns

- **Repository Pattern:** An abstraction layer between the business logic and the data access code. It centralizes query logic and isolates the application from the underlying database framework (e.g., EF Core).
- **Unit of Work Pattern:** Maintains a single database transaction across multiple repositories. It ensures that all data modifications are committed together via `SaveChangesAsync()`, or rolled back completely if any step fails.
- **Dependency Injection (DI):** Decouples application layers by injecting interface abstractions (e.g., `IProductService`) instead of concrete classes, enabling loose coupling and unit testing.

2. Middleware & Global Exception Handling

- **Middleware:** Sequential pipeline components that process incoming HTTP requests and outgoing responses (e.g., routing, authentication, error handling).
- **ExceptionHandler:** A modern, centralized mechanism for catching unhandled exceptions across the application. It logs errors and returns consistent responses without polluting business logic with repetitive try-catch blocks.
- **ProblemDetails (RFC 7807):** A standard machine-readable JSON structure used for API error responses, containing properties like Status, Title, and Detail.

3. EF Core Performance & Indexing

- **\$N+1\$ Query Problem:** Occurs when loading related data in a loop, resulting in 1 initial query plus N additional database calls. Solved by using **Eager Loading** with `.Include()`.
- **AsNoTracking:** Disables EF Core change tracking for read-only queries to reduce memory consumption and execution time.

- **Database Indexes:**

- **Clustered Index:** Sets the physical sorting order of rows on disk (one per table, usually on the Primary Key).
- **Non-Clustered Index:** A separate lookup structure that speeds up queries targeting frequently searched non-key columns.